INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

NoSQL Databases - O2026_ESI3891A

ITESO, Universidad Jesuita de Guadalajara

MongoDB Lab 2 - From Data Model to REST API & Query Optimization

Student:
Luis Mariano Romo Medina , 758007 , IDS : 50%.

Teacher: Leobardo Ruiz

Date: 30/03/2026

Table of Contents

STEP 1 — ADAPT THE DATA MODEL TO AN API **3**

/TEAMS 5
POST *5*
GET *8*
UPDATE *11*
DELETE *13*
/TOURNAMENTS 16
POST *16*
GET *18*
UPDATE *20*
DELETE *23*
/PEOPLE 25
POST *25*
GET *28*
UPDATE *31*
DELETE *34*
/MATCHES 36
POST *36*
GET *39*
UPDATE *41*
DELETE *44*

STEP 2 — EXTEND SEARCH FUNCTIONALITY **46**

STEP 3 — PAGINATION (LIMIT & SKIP) **51**

STEP 4 — INDEX DESIGN AND IMPLEMENTATION **52**

INDEX 1: TEXT INDEX ON TOURNAMENTS NAME. 52
INDEX 2: TEXT INDEX ON TEAM'S NAME. 52
INDEX 3: NUMERIC INDEX ON MATCHES MATCH_DAY. 53

STEP 5 — UPDATE THE CLIENT **54**

STEP 6 — DEMONSTRATE CRUD OPERATIONS **55**

20 INSERTED DOCUMENTS. 55
10 QUERIES (GETS) 55
5 UPDATES 59
5 DELETIONS 60

STEP 7 — REFLECTION: PERFORMANCE & DESIGN **62**

Step 1 — Adapt the Data Model to an API

Operation	Endpoint Example
Create	`POST /resource`
Get all	`GET /resource`
Get by ID	`GET /resource/{id}`
Update	`PUT /resource/{id}`
Delete	`DELETE /resource/{id}`

First of all, this laboratory **enforced multiple changes to** the original schemas designed for each entity (in this case, collections). Regardless of MongoDB or any other NoSQL database being highly flexible, that **doesn't translate into** "accepting anything." The challenges **start to appear** when connecting the backend to the database. Most of the validations are done on the backend side, where the entities are highly typed, so not everything is inserted into the database—only what was intended to be stored in the **collections** (in this case with MongoDB).

So, I will introduce the endpoints designed in the laboratory, but first, a brief explanation of the general structure of the project.

In the original repo, 3 main modules **were given**: main, client, and resources. This was fine because of the small example that was present in it.

But as the project **grew** from 1 (/books) to 4 endpoints (the ones explained in the next steps), a modularization and a different level of structure were needed.

Following a general backend project structure, I decided to separate responsibilities from the business logic (services), which make use of structure validation (validators) and an ORM (Object Relational Mapping) from JSON models to a Python data-modeled structure (models), and the endpoints for each resource inside a (resources) folder.

The workflow in the elaboration of this project was thought as:
Controllers → Services → Validators / Models → DB Manipulation

IMPORTANT CLARIFICATION

There are rules that I defined for how data is **inserted, updated, and deleted** for each resource, but there are validations that **occur** that enforce a certain level of consistency within the documents in the collections where there are references to documents in different collections.

People

- A person can't be created without a `team_id`. Therefore, to insert a new person, at least one team document needs to be present to be linked with the new resource in people.
- A person can't be deleted if it is present as a reference in some field within match documents.

Tournaments

- A tournament can be created with or without initial teams, for later updates.
- A tournament can't be deleted if it is referenced within match documents.

Teams

- A team doesn't have any restriction during the creation phase; meaning it doesn't store references to other documents.
- A team can't be deleted if it is referenced in tournaments, people, or match documents.

Matches

- A match needs to reference 2 teams (home and away) in the creation phase. Therefore, at least 2 documents in teams need to be present before creating a match. Also, a reference to a tournament is required, forcing the creation of a tournament before posting a new match.
- The deletion of a document in the matches collection is always possible and not restricted by any other collection reference.

/Teams

```
app.add_route('/teams', team_resource, suffix="collection")
app.add_route('/teams/{id}', team_resource)
```

Each endpoint regardless of the resource type made use of the same resource that acts as the controller for the named resource in each scenario.

POST

POST endpoint required of data to be passed by the user in each case (tournaments, teams, people, matches) and the service of each resource validate the structure and type correctness expected by the backend before try inserting a new resource to that collection.

```
async def on_post_collection(self, req, res):
    data = await req.media

    team_id = team_service.create_team(data, self.db)
        …
```

Success

Example CLI:

By the terminal the user can either input stats for the new given team or not.

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams create
Creating team...
Enter team name: Manchester City
Enter team country: England
Do you have stats to add? (y/n)
Enter choice: n
"69c7186f3029d03250d9bf74"
```

```
_id: ObjectId('69c7186f3029d03250d9bf74')
name : "Manchester City"
country : "England"
▾ stats : Object
    wins : 0
    draws : 0
    losses : 0
    goals_in_favor : 0
    goals_against : 0
```

Example HTTP:

In the HTTP method, the user is required to pass all the necessary fields for creation.

```
# POST TEAM
POST http://localhost:8000/teams
Content-Type: application/json

{
    "name": "Real Oviedo",
    "country": "Spain",
    "stats": {
        "wins": 4,
        "draws": 9,
        "losses": 16,
        "goals_in_favor": 20,
        "goals_against": 48
    }
}
```

```
  _id: ObjectId('69c719673029d03250d9bf75')
  name : "Real Oviedo"
  country : "Spain"
▾ stats : Object
    wins : 4
    draws : 9
    losses : 16
    goals_in_favor : 20
    goals_against : 48
```

Failures

```
# POST TEAM
POST http://localhost:8000/teams
Content-Type: application/json

{
    "name": "Real Oviedo",
    "country": "Spain"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Missing field",
  "description": "stats is required."
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams create
Creating team...
Enter team name: Pumas
Enter team country:
ERROR: Name and country are required
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams create
Creating team...
Enter team name: Pumas
Enter team country: 33
ERROR: Name and country cannot be numbers
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams create
Creating team...
Enter team name: 8423
Enter team country:
ERROR: Name and country are required
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams create
Creating team...
Enter team name: 434
Enter team country: Mexico
ERROR: Name and country cannot be numbers
```

GET

GET /teams and /teams/{id}

```
#GET ALL
def get_all_teams(db):
    teams = list(db.teams.find())

    for t in teams:
        t["_id"] = str(t["_id"])

    return teams


#GET BY ID
def get_team_by_id(team_id, db):

    try:
        team_id = ObjectId(team_id)
    except InvalidId:
        raise falcon.HTTPBadRequest(title="Invalid ID",
                                    description="Invalid team id")

    team = db.teams.find_one({"_id": team_id})
```

This methods are in themselves self-explanatories, in comparison to other GET methods implemented in different resources where reference between collections exist and some mapping methods are applied so ID are mapped with the name of the reference document, here the requests to the db are done directly, the only validation requirements previous to querying the Database is in the case of getting by ID where a typecast is attempted as verifying that what the user passed is in fact valid ID reference.

Success Gets

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams get --id 69c7207899ba033fd6bb2d62
{
  "_id": "69c7207899ba033fd6bb2d62",
  "name": "Bayern Munich",
  "country": "Germany",
  "stats": {
    "wins": 22,
    "draws": 4,
    "losses": 1,
    "goals_in_favor": 97,
    "goals_against": 25
  }
}
```

Without ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams get
{
  "_id": "69c7207899ba033fd6bb2d62",
  "name": "Bayern Munich",
  "country": "Germany",
  "stats": {
    "wins": 22,
    "draws": 4,
    "losses": 1,
    "goals_in_favor": 97,
    "goals_against": 25
  }
}
{
  "_id": "69c7207899ba033fd6bb2d63",
  "name": "Borussia Dortmund",
  "country": "Germany",
  "stats": {
    "wins": 18,
    "draws": 7,
    "losses": 2,
    "goals_in_favor": 58,
    "goals_against": 28
  }
```

Example HTTP:

With ID

```
###
# GET TEAM BY ID
GET http://localhost:8000/teams/69c7207899ba033fd6bb2d62
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "team": {
    "_id": "69c7207899ba033fd6bb2d62",
    "name": "Bayern Munich",
    "country": "Germany",
    "stats": {
```

Without ID

```
###
# GET ALL TEAMS
GET http://localhost:8000/teams
```

```
GET http://localhost:8000/teams
  Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "teams": [
    {
      "_id": "69c7207899ba033fd6bb2d62",
      "name": "Bayern Munich",
      "country": "Germany",
```

Failure Gets

```
###
# GET TEAM BY WRONG ID
GET http://localhost:8000/teams/69c7207899ba033fd6bb2d6
```

```
GET http://localhost:8000/teams/69c7207899ba033fd6bb2d6
  Show Request

HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid team id"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams get --id 69c7207899ba033fd6bb2d6
Error: 400 - {"title": "Invalid ID", "description": "Invalid team id"}
```

UPDATE

UPDATE /teams/{id}

```
#UPDATE BY ID
def update_team_by_id(team_id, data, db):
    try:
        obj_id = ObjectId(team_id)
    except InvalidId:
        raise falcon.HTTPBadRequest(title="Invalid ID",
                                    description="Invalid team id")

    team_validator.validate_team_data(data, partial=True)

    result = db.teams.update_one({"_id": obj_id}, {"$set": data})

    if result.matched_count == 0:
        raise falcon.HTTPNotFound(description="Team not found")

    team = db.teams.find_one({"_id": obj_id})
    team["_id"] = str(team["_id"])
    return team
```

The only fields allowed to update for a team are “name” and “country”.

Success Updates

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams get --id 69c7d9a3cb266101bdb45278
{
  "_id": "69c7d9a3cb266101bdb45278",
  "name": "FCBayern",
  "country": "Germany",
  "stats": {
    "wins": 22,
    "draws": 4,
    "losses": 1,
    "goals_in_favor": 97,
    "goals_against": 25
  }
}
```

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams update --id 69c7d9a3cb266101bdb45278
Enter new name (or press Enter to skip): FC Bayern Updated
Enter new country (or press Enter to skip):
{
  "_id": "69c7d9a3cb266101bdb45278",
  "name": "FC Bayern Updated",
  "country": "Germany",
  "stats": {
    "wins": 22,
    "draws": 4,
    "losses": 1,
    "goals_in_favor": 97,
    "goals_against": 25
  }
}
```

With Incorrect ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams update --id 69c7d9a3cb266101bdb4527
Enter new name (or press Enter to skip): Updtae
Enter new country (or press Enter to skip): update
Error: 400 - {"title": "Invalid ID", "description": "Invalid team id"}
```

Example HTTP:

With ID

```
# UPDATE TEAM BY ID
PUT http://localhost:8000/teams/69c7d9a3cb266101bdb45278
Content-Type: application/json

{
    "name": "Real Oviedo Updated",
    "country": "Spain"
}
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "team": {
    "_id": "69c7d9a3cb266101bdb45278",
    "name": "Real Oviedo Updated",
    "country": "Spain",
    "stats": {
      "wins": 22,
      "draws": 4,
      "losses": 1,
      "goals_in_favor": 97,
      "goals_against": 25
    }
  }
}
```

With Incorrect ID

```
# UPDATE TEAM BY ID
PUT http://localhost:8000/teams/69c7d9a3cb266101bdb4527
Content-Type: application/json

{
    "name": "Real Oviedo Updated",
    "country": "Spain"
}
```

```
PUT http://localhost:8000/teams/69c7d9a3cb266101bdb4527
  Show Request

HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid team id"
}
```

Failure Updates

```
# UPDATE TEAM BY ID
PUT http://localhost:8000/teams/69c7d9a3cb266101bdb45278
Content-Type: application/json

{
    "name": 32332
}
```

```
PUT http://localhost:8000/teams/69c7d9a3cb266101bdb45278
 Show Request

HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Type",
  "description": "Name must be a non-empty string"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams update --id 69c7d9a3cb266101bdb45278
Enter new name (or press Enter to skip): 9999
Enter new country (or press Enter to skip): Invalid name entered
Error: 400 - {"title": "Invalid Type", "description": "Name must be a non-empty string"}
```

DELETE

DELETE /teams/{id}

```
# DELETE BY ID
def delete_team_by_id(team_id, db):
    try:
        obj_id = ObjectId(team_id)
    except InvalidId:
        raise falcon.HTTPBadRequest(…)

    team = db.teams.find_one({"_id": obj_id})
    if not team:
        raise falcon.HTTPNotFound(description="Team not found")

    # Check if team is in any tournament
    # Check if team has any people associated
    # Check if team is in any match (home_team or away_team)

    # If no references found, proceed with deletion
    db.teams.delete_one({"_id": obj_id})
    team["_id"] = str(team["_id"])
    return team
```

To delete a match is restricted to various conditions, the ones stated here as comments are indicatives of those conditions, that the team is not referenced first in any tournaments, then that a person is also not linked to that team and finally that the team is not present as a reference in either home or away team in matches, this is done for enforcing some level of consistency within the data.

Success Deletes

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams delete --id 69c7d9a3cb266101bdb4529f
Team deleted successfully:
{
  "_id": "69c7d9a3cb266101bdb4529f",
  "name": "Liverpool",
  "country": "England",
  "stats": {
    "wins": 18,
    "draws": 8,
    "losses": 2,
    "goals_in_favor": 65,
    "goals_against": 28
  }
}
```

With Incorrect ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams delete --id 69c7d9a3cb266101bdb4529f
Error: 404 - {"title": "404 Not Found", "description": "Team not found"}
```

Example HTTP:

With ID

```
# DELETE TEAM BY ID
DELETE http://localhost:8000/teams/69c7d9a3cb266101bdb452a1
```

```
HTTP/1.1 200 OK
> {Headers} …content-type: application/json…

{
  "team": {
    "_id": "69c7d9a3cb266101bdb452a1",
    "name": "Arsenal",
    "country": "England",
    "stats": {
      "wins": 16,
      "draws": 8,
      "losses": 4,
      "goals_in_favor": 55,
      "goals_against": 30
    }
  }
}
```

Failure Deletes

```
# DELETE TEAM BY ID
DELETE http://localhost:8000/teams/69c7d9a3cb266101bdb45294
```

```
DELETE http://localhost:8000/teams/69c7d9a3cb266101bdb45294
  Show Request

HTTP/1.1 409 Conflict
> (Headers) …content-type: application/json…

{
  "title": "Cannot Delete",
  "description": "Cannot delete team that is part of a tournament"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams delete --id 69c7d9a3cb266101bdb4527f
Error: 409 - {"title": "Cannot Delete", "description": "Cannot delete team that is part of a tournament"}
```

/Tournaments

```
app.add_route('/tournaments', tournament_resource, suffix="collection")
app.add_route('/tournaments/{id}', tournament_resource)
```

POST

```
# Create POST /
async def on_post_collection(self, req, res):
    data = await req.media
    tournament_id = tournament_service.create_tournament(data, self.db)

    log.info(f"Tournament created: {tournament_id}")

    res.status = falcon.HTTP_201
    res.media = {"tournament": tournament_id}
```

A winner_team may not exist yet at the creation of the tournament so it is allowed to create a tournament that has no winner and, teams in the tournament may still have not been created so it is allowed to create the tournament with an empty array field, for later updates to this field.

Required fields : name, country, total_match_days.

Optional fields : winner team, teams

Success

Example CLI:

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments create
Creating tournament...
Enter tournament name: Ligue One
Enter tournament country: France
Enter total match days: 34
Do you have a winner team id to add? (y/n): n
Do you want to add teams now? (y/n)
Enter choice: n
"69c7e3e5d65391ee1770966a"
```

```
_id: ObjectId('69c7e3e5d65391ee1770966a')
name : "Ligue One"
country : "France"
▸ teams : Array (empty)
total_match_days : 34
winner_team_id : null
```

Example HTTP:

```
# POST TOURNAMENT
POST http://localhost:8000/tournaments
Content-Type: application/json

{
    "name": "Eredivise",
    "country": "Netherlands",
    "total_match_days" : 34
}
```

```
_id: ObjectId('69c7e449d65391ee1770966b')
name : "Eredivise"
country : "Netherlands"
▸ teams : Array (empty)
total_match_days : 34
winner_team_id : null
```

Failures

```
# POST TOURNAMENT
POST http://localhost:8000/tournaments
Content-Type: application/json

{
    "name": "Eredivise",
    "country": "Netherlands"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Missing field",
  "description": "total_match_days is required."
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments create
Creating tournament...
Enter tournament name: Champions League
Enter tournament country: Europe
Enter total match days:
ERROR: Total match days must be an integer
```

GET

GET /tournaments and /tournaments/{id}

```
# Get all  GET /
async def on_get_collection(self, req, res):
    tournaments = tournament_service.get_all_tournaments(self.db)
    res.media = { "tournaments" : tournaments }

# Get by Id  GET /{id}
async def on_get(self, req, res, id):
    tournament = tournament_service.get_tournament_by_id(id, self.db)
    res.media = { "tournament" : tournament }
```

This GET methods retrieve the tournaments either all or by ID, but the reference between the team's collections in a tournament exist via Id reference, so some mapping methods are applied, for getting the names of the teams not the ids in tournaments.

Success Gets

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments get --id 69c7e3d4d65391ee17709616
{
  "_id": "69c7e3d4d65391ee17709616",
  "name": "Premier League",
  "country": "England",
  "teams": [],
  "total_match_days": 38,
  "winner_team_id": null
}
```

Without ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py teams get
{
  "_id": "69c7207899ba033fd6bb2d62",
  "name": "Bayern Munich",
  "country": "Germany",
  "stats": {
    "wins": 22,
    "draws": 4,
    "losses": 1,
    "goals_in_favor": 97,
    "goals_against": 25
  }
}
{
  "_id": "69c7207899ba033fd6bb2d63",
  "name": "Borussia Dortmund",
  "country": "Germany",
  "stats": {
    "wins": 18,
    "draws": 7,
    "losses": 2,
    "goals_in_favor": 58,
    "goals_against": 28
  }
```

Example HTTP:

With ID

```
###
# GET TOURNAMENT BY ID
GET http://localhost:8000/tournaments/69c7e3d4d65391ee17709616
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "tournament": {
    "_id": "69c7e3d4d65391ee17709616",
    "name": "Premier League",
    "country": "England",
    "teams": [],
    "total_match_days": 38,
    "winner_team_id": null
  }
}
```

Without ID

```
# GET ALL TOURNAMENTS
GET http://localhost:8000/tournaments
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "tournaments": [
    {
      "_id": "69c7e3d4d65391ee17709614",
      "name": "Bundesliga",
      "country": "Germany",
```

Failure Gets

```
# GET TOURNAMENT BY WRONG ID
GET http://localhost:8000/tournaments/69c7e3d4d65391ee1770961
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid tournament id"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments get --id 69c7e3d4d65391ee1770961
Error: 400 - {"title": "Invalid ID", "description": "Invalid tournament id"}
```

UPDATE

UPDATE /tournaments/{id}

```
# Update by Id  PUT /{id}
async def on_put(self, req, res, id):
    data = await req.media
    tournament = tournament_service.update_tournament(id, data, self.db)
    log.info(f"Tournament updated: {id}")
    res.media = { "tournament" : tournament}
```

The all fields are allowed to be updated from [name, country, winner, teams, total_match_days]

Success Updates

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments get --id 69c7e3d4d65391ee17709616
{
  "_id": "69c7e3d4d65391ee17709616",
  "name": "Premier League",
  "country": "England",
  "teams": [],
  "total_match_days": 38,
  "winner_team_id": null
}
```

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments update --id 69c7e3d4d65391ee17709616
Enter new name (or press Enter to skip): EPL
Enter new country (or press Enter to skip):
Do you want to update teams? (y/n): n
Do you want to update winner? (y/n): n
{
  "_id": "69c7e3d4d65391ee17709616",
  "name": "EPL",
  "country": "England",
  "teams": [],
  "total_match_days": 38,
  "winner_team_id": null
}
```

With Incorrect ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments update --id 69c7e3d4d65391ee1770961
Enter new name (or press Enter to skip): Premier League
Enter new country (or press Enter to skip): Holand
Do you want to update teams? (y/n): y
Enter team name or id (or press Enter to finish adding teams): 382838484
Enter team name or id (or press Enter to finish adding teams):
Do you want to update winner? (y/n): n
Error: 400 - {"title": "Invalid ID", "description": "Invalid tournament id"}
```

Example HTTP:

With ID

```
# UPDATE TOURNAMENT
PUT http://localhost:8000/tournaments/69c7e3d4d65391ee17709616
Content-Type: application/json

{
    "name": "Eredivisie",
    "country": "Netherlands"
}
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "tournament": {
    "_id": "69c7e3d4d65391ee17709616",
    "name": "Eredivisie",
    "country": "Netherlands",
    "teams": [],
    "total_match_days": 38,
    "winner_team_id": null
  }
}
```

With Incorrect ID

```
# UPDATE TOURNAMENT
PUT http://localhost:8000/tournaments/69c7e3d4d65391ee1770961
Content-Type: application/json

{
    "name": "Eredivisie",
    "country": "Netherlands"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid tournament id"
}
```

Failure Updates

```
# UPDATE TOURNAMENT ( failure )
PUT http://localhost:8000/tournaments/69c7e3d4d65391ee17709616
Content-Type: application/json

{
    "winner_team_id": "64c7e3d4d65391ee17709"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Winner",
  "description": "64c7e3d4d65391ee17709 is not a valid team name or id"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments update --id 69c7e3d4d65391ee1770961
Enter new name (or press Enter to skip): Premier League
Enter new country (or press Enter to skip): ENGLAND
Do you want to update teams? (y/n): y
Enter team name or id (or press Enter to finish adding teams): invalidId
Enter team name or id (or press Enter to finish adding teams):
Do you want to update winner? (y/n): n
Error: 400 - {"title": "Invalid Team", "description": "invalidId is not a valid team name or id"}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments update --id 69c7e3d4d65391ee17709616
Enter new name (or press Enter to skip): EPFL
Enter new country (or press Enter to skip): Nether
Do you want to update teams? (y/n): n
Do you want to update winner? (y/n): y
Enter winner team name/id (or press Enter for None): invalidId
Error: 400 - {"title": "Invalid Winner", "description": "invalidId is not a valid team name or id"}
```

DELETE

DELETE /tournaments/{id}

```
# Delete by Id  DELETE /{id}
async def on_delete(self, req, res, id):
    deleted = tournament_service.delete_tournament(id, self.db)
    if deleted:
        log.info(f"Tournament deleted with id: {id}")
        res.status = falcon.HTTP_200
        res.media = {"tournament": "Tournament deleted successfully"}
    else:
        res.status = falcon.HTTP_404
        res.media = {"error": "Tournament not found"}
```

To delete a tournament is restricted only to if it is referenced by some match document

Success Deletes

Example CLI:

With ID (Not present in any match)

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments delete --id 69c7e3d4d65391ee17709616
Tournament deleted successfully:
"Tournament deleted successfully"
```

With Incorrect ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments delete --id 69c7e3d4d65391ee17709616
Error: 404 - {"title": "404 Not Found", "description": "Tournament not found"}
```

Example HTTP:

With ID

```
# DELETE TOURNAMENT
DELETE http://localhost:8000/tournaments/69c7edcc83d10dec2706a39f
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "tournament": "Tournament deleted successfully"
}
```

Failure Deletes

With ID of a tournament that is present in some match.

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py tournaments delete --id 69c7edcc83d10dec2706a39e
Error: 409 - {"title": "Cannot Delete", "description": "Cannot delete tournament that has matches"}
```

```
# DELETE TOURNAMENT
DELETE http://localhost:8000/tournaments/69c7edcc83d10dec2706a39e
```

```
HTTP/1.1 409 Conflict
> (Headers) …content-type: application/json…

{
  "title": "Cannot Delete",
  "description": "Cannot delete tournament that has matches"
}
```

/People

```
app.add_route('/people', people_resource, suffix="collection")
app.add_route('/people/{id}', people_resource)
```

POST

```
# Create POST /
async def on_post_collection(self, req, res):
    data = await req.media

    people_id = people_service.create_people(data, self.db)

    if not people_id:
        log.error("Failed to create person")
        res.status = falcon.HTTP_400
        res.media = { "error" : "Invalid data" }
        return

    log.info(f"Person created: {people_id}")
    res.status = falcon.HTTP_201
    res.media = { "person" : people_id }
```

A person can be created in two ways, being of type player or manager, something I mentioned in the lab 1, was that this is a discriminator field, so different rules applied in regard of the type of person is created

Player

- Required fields: first_name, last_name, shirt_number
- Optional fields: team_id, attributes

Manager

- Required fields: first_name, last_name
- Optional fields: team_id

Success

Example CLI:

Player

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people create
Creating person...
Do you want to create a player/manager? (Enter 'p' or 'm)
Enter type: p
Enter person's first name: Mariano
Enter person's last name: Romo
Do you want to add a team for this person? (y/n)
Enter choice: n
Enter shirt number: 10
Do you have attributes to add? (y/n)
Enter choice: n
"69c7f6f471dab6682766086b"
```

```
_id: ObjectId('69c7f6f471dab6682766086b')
type : "player"
first_name : "Mariano"
last_name : "Romo"
team_id : null
shirt_number : 10
▸ attributes : Object
```

Manager

```
Do you want to create a player/manager? (Enter 'p' or 'm)
Enter type: m
Enter person's first name: Jurgen
Enter person's last name: Klopp
Do you want to add a team for this person? (y/n)
Enter choice: n
"69c7f73e71dab6682766086c"
```

```
_id: ObjectId('69c7f73e71dab6682766086c')
type : "manager"
first_name : "Jurgen"
last_name : "Klopp"
team_id : null
```

Example HTTP:

Player

```
# POST PEOPLE ( player )
POST http://localhost:8000/people
Content-Type: application/json

{
    "type": "player",
    "first_name": "John",
    "last_name": "Doe",
    "shirt_number": 10
}
```

```
HTTP/1.1 201 Created
> (Headers) …content-type: application/json…

{
  "person": "69c7f7b071dab6682766086d"
}
```

```
_id: ObjectId('69c7f7b071dab6682766086d')
type : "player"
first_name : "John"
last_name : "Doe"
team_id : null
shirt_number : 10
▸ attributes : Object
```

Manager

```
# POST PEOPLE ( manager )
POST http://localhost:8000/people
Content-Type: application/json

{
    "type": "manager",
    "first_name": "Jane",
    "last_name": "Smith"
}
```

```
HTTP/1.1 201 Created
> (Headers) …content-type: application/json…

{
  "person": "69c7f7fb71dab6682766086f"
}
```

```
_id: ObjectId('69c7f7fb71dab6682766086f')
type : "manager"
first_name : "Jane"
last_name : "Smith"
team_id : null
```

Failures

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people create
Creating person...
Do you want to create a player/manager? (Enter 'p' or 'm)
Enter type: p
Enter person's first name:
Enter person's last name:
ERROR: First name and last name cannot be empty
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people create
Creating person...
Do you want to create a player/manager? (Enter 'p' or 'm)
Enter type: p
Enter person's first name: Mariano
Enter person's last name: Romo
Do you want to add a team for this person? (y/n)
Enter choice: y
Enter team name or id: 332332INVALID222222
Enter shirt number: 83
Do you have attributes to add? (y/n)
Enter choice: n
Error: 400 - {"title": "Invalid Team", "description": "332332INVALID222222 is not a valid team name or id"}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people create
Creating person...
Do you want to create a player/manager? (Enter 'p' or 'm)
Enter type: m
Enter person's first name: Vincent
Enter person's last name: Kompany
Do you want to add a team for this person? (y/n)
Enter choice: y
Enter team name or id: INVALID
Error: 400 - {"title": "Invalid Team", "description": "INVALID is not a valid team name or id"}
```

GET

GET /people and /people/{id}

```
# Get all  GET /
async def on_get_collection(self, req, res):
    people = people_service.get_all_people(self.db)
    log.info(f"People: {people}")
    res.media = { "people" : people}


# Get by Id  GET /{id}
async def on_get(self, req, res, id):
    person = people_service.get_people_by_id(id, self.db)
    if person:
        log.info(f"Person: {person}")
        res.media = { "person" : person }
    else:
        log.info(f"Person with id {id} not found")
        res.status = falcon.HTTP_404
        res.media = { "error" : "Person not found" }
```

This GET methods retrieve persons either all or by ID, but the reference between the team's collections in people exists via Id reference, so some mapping methods are applied, for getting the names of the teams not the ids.

Success Gets

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people get --id 69c7f5fa9e4b9caab5ff6cca
{
  "_id": "69c7f5fa9e4b9caab5ff6cca",
  "type": "manager",
  "first_name": "Vincent",
  "last_name": "Kompany",
  "team_id": "Bayern Munich"
}
```

Without ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people get
{
  "_id": "69c7f5fa9e4b9caab5ff6cca",
  "type": "manager",
  "first_name": "Vincent",
  "last_name": "Kompany",
  "team_id": "Bayern Munich"
}
{
  "_id": "69c7f5fa9e4b9caab5ff6ccb",
  "type": "player",
```

Example HTTP:

With ID

```
# GET PEOPLE BY ID ( manager )
GET http://localhost:8000/people/69c7f5fa9e4b9caab5ff6cca
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "person": {
    "_id": "69c7f5fa9e4b9caab5ff6cca",
    "type": "manager",
    "first_name": "Vincent",
    "last_name": "Kompany",
    "team_id": "Bayern Munich"
  }
}
```

Without ID

```
# GET ALL PEOPLE
GET http://localhost:8000/people
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "people": [
    {
      "_id": "69c7f5fa9e4b9caab5ff6cca",
      "type": "manager",
      "first_name": "Vincent",
      "last_name": "Kompany",
      "team_id": "Bayern Munich"
    },
    {
```

Failure Gets

```
# GET PEOPLE BY wrong ID
GET http://localhost:8000/people/69c7f5fa9e4b9caab5ff6cc
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid people id"
}
```

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people get --id 69c7f5fa9e4b9caab5ff6cc
Error: 400 - {"title": "Invalid ID", "description": "Invalid people id"}
```

UPDATE

UPDATE /people/{id}

```
# Update by Id  PUT /{id}
async def on_put(self, req, res, id):
    try:
        data = await req.media
        result = people_service.update_people_by_id(id, data, self.db)
        res.media = {"person": result}
    except ValueError as e:
        log.error(f"Failed to update person: {e}")
        res.status = falcon.HTTP_400
        res.media = {"error": str(e)}
```

A type in person cannot be changed, but other aspects can.

- Team id can be updated for both players and managers
- Only a player can change the shirt_number attribute and attribute “attributes” because only he, has them

Success Updates

Example CLI:

With ID (player)

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people update --id 69c7f5fb9e4b9caab5ff6d05
Are you updating a player or a manager? (Enter 'p' or 'm'):
p
Updating a player. You can update team, shirt number, and attributes.
New team name/id (or press Enter to skip):
New shirt number (or press Enter to skip): 99
Do you want to update attributes? (y/n):
n
{
  "_id": "69c7f5fb9e4b9caab5ff6d05",
  "type": "player",
  "first_name": "Giorgi",
  "last_name": "Mamardashvili",
  "team_id": "Valencia",
  "shirt_number": 99,
  "attributes": {
    "reflexes": 85,
    "positioning": 84,
    "distribution": 83
  }
}
```

With ID (manager)

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people update --id 69c7f5fa9e4b9caab5ff6cca
Are you updating a player or a manager? (Enter 'p' or 'm'):
m
Updating a manager. You can update team.
New team name/id (or press Enter to skip): 69c7f5fa9e4b9caab5ff6ca3
{
  "_id": "69c7f5fa9e4b9caab5ff6cca",
  "type": "manager",
  "first_name": "Vincent",
  "last_name": "Kompany",
  "team_id": "SC Freiburg"
}
```

Example HTTP:

With Player ID

```
# UPDATE PEOPLE ( player )
PUT http://localhost:8000/people/69c7f7b071dab6682766086d
Content-Type: application/json

{
    "shirt_number": 7
}
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "person": {
    "_id": "69c7f7b071dab6682766086d",
    "type": "player",
    "first_name": "John",
    "last_name": "Doe",
    "team_id": "None",
    "shirt_number": 7,
    "attributes": {}
  }
}
```

With Manager ID

```
# UPDATE PEOPLE ( manager )
PUT http://localhost:8000/people/69c7f7f771dab6682766086e
Content-Type: application/json

{
    "team_id": "69c7f5fa9e4b9caab5ff6ca4"
}
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "person": {
    "_id": "69c7f7f771dab6682766086e",
    "type": "manager",
    "first_name": "Jane",
    "last_name": "Smith",
    "team_id": "1. FC Union Berlin"
  }
}
```

Failure Updates

Trying to update a player with the wrong attributes

```
# UPDATE PEOPLE ( player )
PUT http://localhost:8000/people/69c7f7b071dab6682766086d
Content-Type: application/json

{
  "first_name" : "INVALID UPDATE CAMP",
  "last_name" : "INVALID UPDATE CAMP"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Field",
  "description": "Cannot update 'first_name'. Only team_id, shirt_number, and attributes can be updated for players, or team_id for managers"
}
```

Trying to update a player with an invalid new team_id

```
# UPDATE PEOPLE ( player )
PUT http://localhost:8000/people/69c7f7b071dab6682766086d
Content-Type: application/json

{
  "team_id" : "INVALID UPDATE CAMP"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Team",
  "description": "INVALID UPDATE CAMP is not a valid team name or id"
}
```

Trying to update a manager with an invalid new team_id

```
# UPDATE PEOPLE ( manager )
PUT http://localhost:8000/people/69c7f7f771dab6682766086e
Content-Type: application/json

{
    "team_id": "69c7f5fa9e4b9caab5ffa4"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Team",
  "description": "69c7f5fa9e4b9caab5ffa4 is not a valid team name or id"
}
```

Trying to update a manager with an invalid camp

```
# UPDATE PEOPLE ( manager )
PUT http://localhost:8000/people/69c7f7f771dab6682766086e
Content-Type: application/json

{
    "invalid_camp": "69c7f5fa9e4b9caab5ffa4"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid Field",
  "description": "Cannot update 'invalid_camp'. Only team_id, shirt_number, and attributes can be updated for players, or team_id for managers"
}
```

DELETE

DELETE /people/{id}

```
# Delete by Id  DELETE /{id}
async def on_delete(self, req, res, id):
    result = people_service.delete_people_by_id(id, self.db)
    if result:
        log.info(f"Person with id {id} deleted")
        res.media = {"people": "Person deleted"}
    else:
        log.info(f"Person with id {id} not found for deletion")
        res.status = falcon.HTTP_404
        res.media = {"error": "Person not found"}
```

To delete a person regardless of the type, it needs to not be present in the matches collections as a reference attribute.

Success Deletes

Example CLI:

With ID (Not present in any match)

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people delete --id 69c7f73e71dab6682766086c
Person deleted successfully:
"Person deleted"
```

Example HTTP:

With ID

```
# DELETE PEOPLE
DELETE http://localhost:8000/people/69c7f7f771dab6682766086e
```

```
HTTP/1.1 200 OK
>(Headers) …content-type: application/json…

{
  "people": "Person deleted"
}
```

Failure Deletes

With ID of a person that is present in some match.

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py people delete --id 69c7f5fa9e4b9caab5ff6ce0
Error: 409 - {"title": "Cannot Delete", "description": "Cannot delete person who is MVP in a match"}
```

/Matches

```
app.add_route('/matches', match_resource, suffix="collection")
app.add_route('/matches/{id}', match_resource)
```

POST

```
# Create POST /
async def on_post_collection(self, req, res):
    data = await req.media

    match_id = match_service.create_match(data, self.db)

    res.status = falcon.HTTP_201
    res.media = {"match": match_id}
```

A person can be created in two ways, being of type player or manager, something I mentioned in the lab 1, was that this is a discriminator field, so different rules applied in regard of the type of person is created

- Required fields: all
- Optional fields: none

For this resource, is required everything is provided and there's no escaping from this.

Success

Example CLI:

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches create
Creating match...
Enter tournament name or id: Bundesliga
Enter match day (integer): 32
Enter match date (e.g., 2024-01-13): 2026-03-26

--- Home Team ---
Enter home team name or id: Bayern Munich
Enter home team name (for display): Bayern Munich
Enter goals scored by home team: 5

--- Away Team ---
Enter away team name or id: Borussia Dortmund
Enter away team name (for display): BVB
Enter goals scored by away team: 0
Enter MVP player (FirstName LastName or id): Manuel Neuer

--- Match Statistics ---
Possession %:
  Home: 67
  Away: 33
Shots on target:
  Home: 24
  Away: 10
Corners:
  Home: 18
  Away: 3
Fouls:
  Home: 4
  Away: 19

--- Player Statistics ---
Add player stats? (y/n): n
"69c807ee6b195b9579bbca99"
```

Example HTTP:

```
# POST MATCHES
###
POST http://localhost:8000/matches
Content-Type: application/json

{
  "tournament_id": "Bundesliga",
  "match_day": 32,
  "date": "2026-03-26",
  "home_team": {
    "team_id": "Bayern Munich",
    "name": "Bayern Munich",
    "goals": 5
  },
  "away_team": {
    "team_id": "Borussia Dortmund",
    "name": "BVB",
    "goals": 0
  },
  "mvp_player_id": "Manuel Neuer",
  "stats": {
    "possession": {
      "home": 67,
      "away": 33
    },
    "shots_on_target": {
      "home": 24,
      "away": 10
    },
    "corners": {
      "home": 18,
      "away": 3
    },
    "fouls": {
      "home": 4,
      "away": 19
    }
  },
  "player_stats": []
}
```

```
HTTP/1.1 201 Created
> (Headers) …content-type: application/json…

{
  "match": "69c8098c6b195b9579bbca9a"
}
```

Failures

By no means the attempt to fail in creating a match resource is exhaustive as there are many permutations on how a match can be incorrectly created but here are some examples.

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches create
Creating match...
Enter tournament name or id: Invalid Tournament
Enter match day (integer): 3
Enter match date (e.g., 2024-01-13): 2222

--- Home Team ---
Enter home team name or id: None
Enter home team name (for display):
Enter goals scored by home team:
ERROR: All home team fields are required, goals must be integer
```

```
Enter match date (e.g., 2024-01-13): 2222

--- Home Team ---
Enter home team name or id: None
Enter home team name (for display):
Enter goals scored by home team:
ERROR: All home team fields are required, goals must be integer
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches create
Creating match...
Enter tournament name or id: Bundesliga
Enter match day (integer): 45
Enter match date (e.g., 2024-01-13): 33

--- Home Team ---
Enter home team name or id: INVALID
Enter home team name (for display): INVALID
Enter goals scored by home team: 3

--- Away Team ---
Enter away team name or id: Invalid
Enter away team name (for display): Invalid
Enter goals scored by away team: 3
Enter MVP player (FirstName LastName or id): Invalid

--- Match Statistics ---
Possession %:
  Home: 3
  Away: 4
Shots on target:
  Home: 2
  Away: 4
Corners:
  Home: 4
  Away: 2
Fouls:
  Home: 4
  Away: 3

--- Player Statistics ---
Add player stats? (y/n): n
Error: 400 - {"title": "Invalid Team", "description": "INVALID is not a valid team name or id"}
```

GET

GET /matches and /matches/{id}

```
# Get all  GET /
async def on_get_collection(self, req, res):
    matches = match_service.get_all_matches(self.db)
    log.info(f"Matches: {matches}")
    res.media = {"matches": matches}

# Get by Id  GET /{id}
async def on_get(self, req, res, id):
    match = match_service.get_match_by_id(id, self.db)
    if match:
        log.info(f"Match: {match}")
        res.media = {"match": match}
    else:
        res.status = falcon.HTTP_404
        res.media = {"error": "Match not found"}
```

This GET methods retrieve matches and replace the fields where and Id reference will be stored with the name of the reference, for a better user experience.

Success Gets

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches get --id 69c807646b195b9579bbca96
{
  "_id": "69c807646b195b9579bbca96",
  "tournament_id": "La Liga",
  "match_day": 1,
  "date": "2024-01-14",
  "home_team": {
    "team_id": "Barcelona",
    "name": "Barcelona",
    "goals": 4
  },
  "away_team": {
    "team_id": "Real Madrid",
```

Without ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches get
{
  "_id": "69c807646b195b9579bbca94",
  "tournament_id": "Bundesliga",
  "match_day": 1,
  "date": "2024-01-13",
  "home_team": {
    "team_id": "Bayern Munich",
```

Example HTTP:

With ID

```
### GET MATCH BY ID
GET http://localhost:8000/matches/69c807646b195b9579bbca96
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "match": {
    "_id": "69c807646b195b9579bbca96",
    "tournament_id": "La Liga",
    "match_day": 1,
```

Without ID

```
# GET MATCHES
GET http://localhost:8000/matches
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c807646b195b9579bbca94",
      "tournament_id": "Bundesliga",
      "match_day": 1,
```

Failure Gets

```
# GET PEOPLE BY wrong ID
GET http://localhost:8000/matches/69c7f5fa9e4b9caab5ff6cc
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid match id"
}
```

UPDATE

UPDATE /matches/{id}

```
# Update by Id  PUT /{id}
async def on_put(self, req, res, id):
    data = await req.media
    updated_match = match_service.update_match(id, data, self.db)
    if updated_match:
        log.info(f"Updated Match: {updated_match}")
        res.media = {"match": updated_match}
    else:
        res.status = falcon.HTTP_404
        res.media = {"error": "Match not found"}
```

A match can only be updated by the fields (mvp player, match stats, players stats and match date) this was decided on top of time constraints, because allowing a user to update other fields would have increased the complexity and the delivery would have been put in risk.

Success Updates

Example CLI:

With ID

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches update --id 69c8898c6b195b9579bbca9a
What do you want to update? (Choose one or more)
1. MVP player
2. Match statistics
3. Player statistics
4. Match date
Update MVP player? (y/n): y
Enter new MVP player name/id: Serge Gnabry
Update match statistics? (y/n): n
Update player statistics? (y/n): n
Update match date? (y/n): n
{
  "_id": "69c8898c6b195b9579bbca9a",
  "tournament_id": "Bundesliga",
  "match_day": 32,
  "date": "2026-03-26",
  "home_team": {
    "team_id": "Bayern Munich",
    "name": "Bayern Munich",
    "goals": 5
  },
  "away_team": {
    "team_id": "Borussia Dortmund",
    "name": "BVB",
    "goals": 0
  },
  "mvp_player_id": "Serge Gnabry",
```

Example HTTP:

With Player ID

```
# UPDATE MATCH
PUT http://localhost:8000/matches/69c8098c6b195b9579bbca9a
Content-Type: application/json

{
  "mvp_player_id" : "Manuel Neuer"
}
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "match": {
    "_id": "69c8098c6b195b9579bbca9a",
    "tournament_id": "Bundesliga",
    "match_day": 32,
    "date": "2026-03-26",
    "home_team": {
      "team_id": "Bayern Munich",
      "name": "Bayern Munich",
      "goals": 5
    },
    "away_team": {
      "team_id": "Borussia Dortmund",
      "name": "BVB",
      "goals": 0
    },
    "mvp_player_id": "Manuel Neuer",
    "stats": {
```

Failure Updates

Trying to update a match with 3 different invalid fields or data types.

```
# UPDATE WITH INVALID ID FORMAT
PUT http://localhost:8000/matches/69c8098c6b195b9579bbca9a
Content-Type: application/json

{
    "mvp_player_id": "Robert Lewandowski"
}
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid MVP Player",
  "description": "Robert Lewandowski is not a valid player name or id"
}
```

```
# UPDATE WITH NON-EXISTENT ID
PUT http://localhost:8000/matches/000000000000000000000000
Content-Type: application/json

{
    "goals": -5
}
```

```
HTTP/1.1 404 Not Found
> (Headers) …content-type: application/json…

{
  "title": "404 Not Found",
  "description": "Match not found"
}
```

```
# UPDATE WITH INVALID FIELD
PUT http://localhost:8000/matches/69c8098c6b195b9579bbca9a
Content-Type: application/json

{
    "invalid_field": "some_value",
    "another_wrong_field": 123
}
```

```
HTTP/1.1 404 Not Found
> (Headers) …content-type: application/json…

{
  "title": "404 Not Found",
  "description": "Match not found"
}
```

DELETE

DELETE /matches/{id}

```
# Delete by Id  DELETE /{id}
async def on_delete(self, req, res, id):
    deleted = match_service.delete_match(id, self.db)
    if deleted:
        log.info(f"Deleted Match with id: {id}")
        res.status = falcon.HTTP_200
        res.media = {"match": "Match deleted successfully"}
    else:
        res.status = falcon.HTTP_404
        res.media = {"error": "Match not found"}
```

To delete a match there are not restrictions, as no person, tournament or team references a match resource.

Success Deletes

Example CLI:

```
(.venv) mrmariano@8CN1120MPK mongo-lab2 % python client.py matches delete --id 69c807ee6b195b9579bbca99
Match deleted successfully:
"Match deleted successfully"
```

Example HTTP:

```
# DELETE MATCH
DELETE http://localhost:8000/matches/69c807646b195b9579bbca98
```

```
HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "match": "Match deleted successfully"
}
```

Failure Deletes

Incorrect ID

```
###
# DELETE MATCH
DELETE http://localhost:8000/matches/69c807646b195b9579bbca
```

```
HTTP/1.1 400 Bad Request
> (Headers) …content-type: application/json…

{
  "title": "Invalid ID",
  "description": "Invalid match id"
}
```

Step 2 — Extend Search Functionality

The decision on what endpoint the extended query with 4 parameters was going to be performed, was on the matches collections, as a match resource has the most variety in fields, allowing to make queries parametrized (filter by different text or numeric fields).

A GET/ endpoint for matches was implemented to support advanced filtering through query parameters allowing users to consult specific matches without the need to retrieving all the registries.

Supported Query Parameters

- Numeric with comparators

  Match Day
  - Match_day = ? (exact)
  - match_day_min=?&match_day_max=(Range match_day >= 1 AND match_day <= 3)

  Goals
  - home_goals=3 (exact)
  - home_goals_min=2&home_goals_max=5 (range)
  - away_goals=2 (exact)
  - away_goals_min=1&away_goals_max=3 (range)

- Text parameters (partial and case-insensitive)

  Tournament
  - tournament=Bundesliga (partial search)
    The search is case insensitive so the search for “Bundesliga”, “BUNDESLIGA”, “Bundesliga” returns the same results.

  MVP Player
  - mvp=Musiala (partial search)
  - mvp=Gna (finds "Gnabry")

Internally, it searches in the people table by first_name or last_name letters that fits, then it resolves the ObjectId of the found players for filtering the matches.

Benefits

1. Reducing the network traffic, instead of returning a thousand matches, the end user only receives the 10 he is most interested in.
2. Upgrade in performance, as MongoDB process the filtering directly, and is not performed by the application.
3. Flexibility, the user a has control in exactly what data he receives.

Examples

Query 1: Obtains all matches (without filters)

```
###
#GET /matches
Response:
{
  "matches": [
    { "tournament_id": "Bundesliga", "match_day": 1, ... },
    { "tournament_id": "La Liga", "match_day": 1, ... },
    ...
  ]
}
```

Query 2: Filter by exact day

```
#GET /matches
GET http://localhost:8000/matches?match_day=1
```

```
GET http://localhost:8000/matches?match_day=1
 Show Request

HTTP/1.1 200 OK
(Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b321",
      "tournament_id": "Bundesliga",
      "match_day": 1,
```

Query 3: Range of days

```
#GET /matches
GET http://localhost:8000/matches?match_day_min=1&match_day_max=4
```

```
GET http://localhost:8000/matches?match_day_min=1&match_day_max=4
 Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b321",
      "tournament_id": "Bundesliga",
      "match_day": 1,
},
{
  "_id": "69c838ede8f749dcb391b323",
  "tournament_id": "La Liga",
  "match_day": 1,
```

```
},
{
  "_id": "69c838ede8f749dcb391b322",
  "tournament_id": "Bundesliga",
  "match_day": 2,
```

Query 4: filter by goals (home_goals >= 3)

```
#GET /matches
GET http://localhost:8000/matches?home_goals_min=5
```

```
GET http://localhost:8000/matches?home_goals_min=5
 Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b324",
      "tournament_id": "Bundesliga",
      "match_day": 3,
      "date": "2024-01-27",
      "home_team": {
        "team_id": "Bayern Munich",
        "name": "Bayern Munich",
        "goals": 5
      },
```

Query 5: search by tournament

```
#GET /matches
GET http://localhost:8000/matches?tournament=la%20liga
```

```
GET http://localhost:8000/matches?tournament=la liga
Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b323",
      "tournament_id": "La Liga",
      "match_day": 1,
      "date": "2024-01-14",
      "home_team": {
        "team_id": "Barcelona",
        "name": "Barcelona",
        "goals": 4
```

Query 6: Search by MVP

```
#GET /matches
GET http://localhost:8000/matches?mvp=jamal
```

```
GET http://localhost:8000/matches?mvp=jamal
Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b321",
      "tournament_id": "Bundesliga",
      "match_day": 1,
      "date": "2024-01-13",
      "home_team": {
        "team_id": "Bayern Munich",
        "name": "Bayern Munich",
        "goals": 3
      },
      "away_team": {
        "team_id": "Borussia Dortmund",
        "name": "Borussia Dortmund",
        "goals": 2
      },
      "mvp_player_id": "Jamal Musiala",
```

Query 8: Combined filters

```
#GET /matches
GET http://localhost:8000/matches?mvp=king&home_goals_min=3&match_day=3&tournament=bundesliga
```

```
GET http://localhost:8000/matches?mvp=king&home_goals_min=3&match_day=3&tournament=bundesliga
 Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b324",
      "tournament_id": "Bundesliga",
      "match_day": 3,
      "date": "2024-01-27",
      "home_team": {
        "team_id": "Bayern Munich",
        "name": "Bayern Munich",
        "goals": 5
      },
      "away_team": {
        "team_id": "VfB Stuttgart",
        "name": "VfB Stuttgart",
        "goals": 1
      },
      "mvp_player_id": "Kingsley Coman",
```

Technical Details.

1. The search in tournaments utilizes the $regex option with the “I” option that is for case insensitive matching
2. Mvp search filtering , searches in people.first_name and .last_name insensitively of casing, extracts the ObjectId and finally filters the matches by $in option using the formed list of ObjectId of players.

Step 3 — Pagination (Limit & Skip)

```python
# Get all  GET /
async def on_get_collection(self, req, res):
    """GET /matches - Get all matches with filters and pagination"""
    try:
        # Parse parameters
        limit, skip = match_service.parse_pagination(req)
        params = match_service.parse_filter_params(req)

        # Build filter
        query_filter = match_service.build_match_filter(params, self.db)
        if query_filter is None:
            res.media = {"matches": [],
                         "limit": limit, "skip": skip,
                         "total": 0, "returned": 0}
            return

        # Execute query
        total_count = self.db.matches.count_documents(query_filter)
        matches = list(self.db.matches
                .find(query_filter)
                .skip(skip)
                .limit(limit))

        ….
```

The pagination a limit is done through the methods provided by the MongoDB client, as it has built in skip and limit methods, the only challenge relies on parsing the query parameters of limit and skip to integers and ensure the parameters are rightly requested as (no negatives, no over 100 in limit).

```
#GET /matches
GET http://localhost:8000/matches?limit=1&skip=1
```

```
GET http://localhost:8000/matches?limit=1&skip=1
 Show Request

HTTP/1.1 200 OK
> (Headers) …content-type: application/json…

{
  "matches": [
    {
      "_id": "69c838ede8f749dcb391b322",
      "tournament_id": "Bundesliga",
      "match_day": 2,
```

```
      ]
    }
  ],
  "limit": 1,
  "skip": 1,
  "total": 5,
  "returned": 1
}
Response file saved.
> 2026-03-28T150448.200.json
```

Step 4 — Index Design and Implementation

Index 1: Text Index on tournaments name.

Command:

```
db.tournaments.createIndex([("name", "text")])
```

Query Supported:

GET /matches?tournament=Bundesliga
db.tournaments.find({"name": {"$regex": "Bundesliga", "$options": "i"}})

Why It Improves Performance:

Without this index, the application had to perform a full collection scan on the tournaments collection to find matches by tournament name. The text index enables partial string matching at the database level, eliminating the need for the application to maintain an in-memory mapping of tournament names to ObjectIds.

Index 2: Text Index on team's name.

Command:

```
db.teams.createIndex([("name", "text")])
```

Query Supported:

GET /teams?name=Bayern
db.teams.find({"name": {"$regex": "Bayern", "$options": "i"}})

Why It Improves Performance:

Similarly to tournaments, the team's collection requires frequent name-based lookups during match creation and filtering. The text index enables case-insensitive partial matching without collection-wide scans. This reduces database query time and decreases memory overhead in the application layer.

Index 3: Numeric Index on matches Match_day.

Command:

```
db.matches.createIndex([("match_day", 1)])
```

Query Supported:

GET /matches?match_day=1
GET /matches?match_day_min=1&match_day_max=3
db.matches.find({"match_day": {"$gte": 1, "$lte": 3}})

Why It Improves Performance:

Match day is a frequently queried numeric field that supports both exact matches and range queries (greater than, less than). Without an index, MongoDB must scan every match document to find records matching the criteria, but with an index on match_day now efficient range scans and direct lookups can be performed. This index is crucial for performance as the matches collection grows, especially when filtering by match day ranges.

Step 5 — Update the Client

```
parser.add_argument("--limit", type=int)
parser.add_argument("--skip", type=int)
```

```
if args.action == "search":
    if args.resource != "matches":
        print("ERROR: Search action is only supported for matches resource")
        exit(1)

    search_params = {}
    if args.search_params:
        for param in args.search_params:
            if "=" in param:
                key, value = param.split("=", 1)
                search_params[key] = value
            else:
                print(f"ERROR: Invalid search parameter format:
                             {param}. Expected format is key=value.")
                exit(1)

    if args.limit is not None:
        search_params["limit"] = args.limit
    if args.skip is not None:
        search_params["skip"] = args.skip

    api.search_matches(search_params)
```

If a search wants to be performed in other than the matches collections, the program exits with an error because is only with the matches collections where a search can be performed passing query parameters, in essence, only there are validations and parsing of the arguments, because the implementation was already done in the previous step.

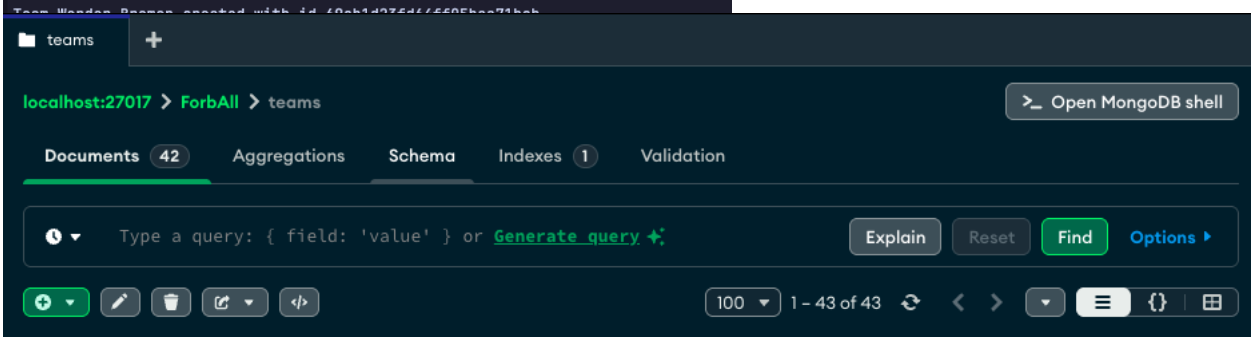
Example of execution

```
(.venv) mrmariano@MacBook-Pro-Mariano mongo-lab2 % python client.py matches search --limit 3
{
  "_id": "69c838ede8f749dcb391b321",
  "tournament_id": "Bundesliga",
  "match_day": 1,
  "date": "2024-01-13",
  "home_team": {
    "team_id": "Bayern Munich",
    "name": "Bayern Munich",
    "goals": 3
  },
  "away_team": {
    "team_id": "Borussia Dortmund",
    "name": "Borussia Dortmund",
    "goals": 2
  },
```

Step 6 — Demonstrate CRUD Operations

20 Inserted Documents.

```
(.venv) mrmariano@MacBook-Pro-Mariano mongo-lab2 % python client.py --reset
This will DELETE all data. Are you sure? (yes/no): yes
Database reset successfully.
(.venv) mrmariano@MacBook-Pro-Mariano mongo-lab2 % python client.py --load
Team Bayern Munich created with id 69cb1d23fd64ff05baa71bbe
Team Borussia Dortmund created with id 69cb1d23fd64ff05baa71bbf
Team VfB Stuttgart created with id 69cb1d23fd64ff05baa71bc0
Team RB Leipzig created with id 69cb1d23fd64ff05baa71bc1
Team TSG Hoffenheim created with id 69cb1d23fd64ff05baa71bc2
Team Bayer Leverkusen created with id 69cb1d23fd64ff05baa71bc3
Team Eintracht Frankfurt created with id 69cb1d23fd64ff05baa71bc4
Team SC Freiburg created with id 69cb1d23fd64ff05baa71bc5
Team 1. FC Union Berlin created with id 69cb1d23fd64ff05baa71bc6
Team FC Augsburg created with id 69cb1d23fd64ff05baa71bc7
Team Mainz created with id 69cb1d23fd64ff05baa71bc8
Team Hamburg SV created with id 69cb1d23fd64ff05baa71bc9
Team Borussia Monchengladbach created with id 69cb1d23fd64ff05baa71bca
```



10 Queries (Gets)

1. Get the resource of the Bayern Team

```
ForbAll> db.teams.find({ name : "Bayern Munich"}, { _id : 1, name: 1})
[
  { _id: ObjectId('69cb1d23fd64ff05baa71bbe'), name: 'Bayern Munich' }
]
```

2. Get the matches that were played on the matchday 2

```
ForbAll> db.matches.find({match_day : 2 }, {_id : 1})
[
  { _id: ObjectId('69cb1d23fd64ff05baa71c3b') },
  { _id: ObjectId('69cb1d23fd64ff05baa71c3e') }
]
```

3. Get the matches where the MVP was Jamal

```
ForbAll> db.matches.find(
  { mvp_player_id: ObjectId("69cb1d23fd64ff05baa71bf0") },
  { _id: 1, mvp_player_id: 1 }
)
> [
  {
    _id: ObjectId('69cb1d23fd64ff05baa71c3a'),
    mvp_player_id: ObjectId('69cb1d23fd64ff05baa71bf0')
  }
]
```

4. Get the tournaments that are played in Germany

```
ForbAll> db.tournaments.find({ country : "Germany"})
[
  {
    _id: ObjectId('69cb1d23fd64ff05baa71be9'),
    name: 'Bundesliga',
    country: 'Germany',
    teams: [
      ObjectId('69cb1d23fd64ff05baa71bbe'),
      ObjectId('69cb1d23fd64ff05baa71bbf'),
      ObjectId('69cb1d23fd64ff05baa71bc0'),
      ObjectId('69cb1d23fd64ff05baa71bc1'),
      ObjectId('69cb1d23fd64ff05baa71bc2'),
      ObjectId('69cb1d23fd64ff05baa71bc3'),
      ObjectId('69cb1d23fd64ff05baa71bc4'),
      ObjectId('69cb1d23fd64ff05baa71bc5'),
      ObjectId('69cb1d23fd64ff05baa71bc6'),
      ObjectId('69cb1d23fd64ff05baa71bc7'),
      ObjectId('69cb1d23fd64ff05baa71bc8'),
      ObjectId('69cb1d23fd64ff05baa71bc9'),
      ObjectId('69cb1d23fd64ff05baa71bca'),
      ObjectId('69cb1d23fd64ff05baa71bcb'),
      ObjectId('69cb1d23fd64ff05baa71bcc'),
      ObjectId('69cb1d23fd64ff05baa71bcd'),
      ObjectId('69cb1d23fd64ff05baa71bce'),
      ObjectId('69cb1d23fd64ff05baa71bcf')
    ],
    total_match_days: 34,
    winner_team_id: null
  }
]
```

5. Get the teams with more than 15 wins

```
ForbAll> db.teams.find({"stats.wins" : { "$gt" : 15 } }, { _id : 0, name : 1,
"stats.wins" : 1 })
[
  { name: 'Bayern Munich', stats: { wins: 22 } },
  { name: 'Borussia Dortmund', stats: { wins: 18 } },
  { name: 'VfB Stuttgart', stats: { wins: 16 } },
  { name: 'Barcelona', stats: { wins: 24 } },
  { name: 'Real Madrid', stats: { wins: 22 } },
  { name: 'Villarreal', stats: { wins: 18 } },
  { name: 'Atletico de Madrid', stats: { wins: 17 } },
  { name: 'Manchester United', stats: { wins: 20 } },
  { name: 'Liverpool', stats: { wins: 18 } },
  { name: 'Chelsea', stats: { wins: 17 } },
  { name: 'Arsenal', stats: { wins: 16 } }
  { name: 'Manchester City', stats: { wins: 22 } }
```

6. Get the players that play for Bayern

```
ForbAll> db.people.find({ type : "player", team_id :
ObjectId("69cb1d23fd64ff05baa71bbe")}, {_id : 1 , first_name : 1, type : 1})
[
  {
    _id: ObjectId('69cb1d23fd64ff05baa71bed'),
    type: 'player',
    first_name: 'Manuel'
  },
  {
    _id: ObjectId('69cb1d23fd64ff05baa71bee'),
    type: 'player',
    first_name: 'Dayot'
  },
  {
```

```
    _id: ObjectId('69cb1d23fd64ff05baa71bef'),
    type: 'player',
    first_name: 'Alphonso'
  },
  {
    _id: ObjectId('69cb1d23fd64ff05baa71bf0'),
    type: 'player',
    first_name: 'Jamal'
  },
  {
    _id: ObjectId('69cb1d23fd64ff05baa71bf1'),
    type: 'player',
    first_name: 'Kingsley'
  },
  {
    _id: ObjectId('69cb1d23fd64ff05baa71bf2'),
    type: 'player',
    first_name: 'Serge'
  }
]
```

7. Get the matches where the away team scored at least 2 goals.

```
ForbAll> db.matches.find({ "away_team.goals" : { "$gte" : 2 }}, { _id : 0 , "away_team" :
1 , "home_team" : 1})
[
  {
    home_team: {
      team_id: ObjectId('69cb1d23fd64ff05baa71bbe'),
      name: 'Bayern Munich',
      goals: 3
    },
    away_team: {
      team_id: ObjectId('69cb1d23fd64ff05baa71bbf'),
      name: 'Borussia Dortmund',
      goals: 2
    }
  },
  {
    home_team: {
      team_id: ObjectId('69cb1d23fd64ff05baa71bd3'),
      name: 'Atletico de Madrid',
      goals: 2
    },
    away_team: {
      team_id: ObjectId('69cb1d23fd64ff05baa71bd2'),
      name: 'Villarreal',
      goals: 2
    }
  }
]
```

8. Get all the managers.

```
ForbAll> db.people.find({ type : "manager" } , { _id : 0 , first_name : 1, type : 1})
[
  { type: 'manager', first_name: 'Vincent' },
  { type: 'manager', first_name: 'Nuri' },
  { type: 'manager', first_name: 'Sebastian' },
  { type: 'manager', first_name: 'Marco' },
  { type: 'manager', first_name: 'Luis' },
  { type: 'manager', first_name: 'Carlo' },
  { type: 'manager', first_name: 'Marcelino' },
  { type: 'manager', first_name: 'Luis' },
  { type: 'manager', first_name: 'Manuel' },
  { type: 'manager', first_name: 'Claudio' },
  { type: 'manager', first_name: 'Imanol' },
  { type: 'manager', first_name: 'José' },
  { type: 'manager', first_name: 'Fernando' },
  { type: 'manager', first_name: 'Jagoba' },
  { type: 'manager', first_name: 'Luis' },
  { type: 'manager', first_name: 'Rubén' },
  { type: 'manager', first_name: 'Michel' },
  { type: 'manager', first_name: 'Iñigo' },
  { type: 'manager', first_name: 'Javier' },
  { type: 'manager', first_name: 'Luis' }
]
```

9. Get all the matches played in the “La Liga” tournament.

```
ForbAll> db.tournaments.find( { name : "La Liga"}, { _id : 1 })
[ { _id: ObjectId('69cb1d23fd64ff05baa71bea') } ]
ForbAll> db.matches.find({ tournament_id :
ObjectId("69cb1d23fd64ff05baa71bea")} , { tournament_id : 1 , _id : 1})
[
  {
    _id: ObjectId('69cb1d23fd64ff05baa71c3c'),
    tournament_id: ObjectId('69cb1d23fd64ff05baa71bea')
  },
  {
    _id: ObjectId('69cb1d23fd64ff05baa71c3e'),
    tournament_id: ObjectId('69cb1d23fd64ff05baa71bea')
  }
]
```

10. Get all the matches where the local team didn’t score.

```
ForbAll> db.matches.find({ "home_team.goals" : { "$eq" : 0 }} , {
home_team : 1})
```

5 Updates

1. Update Bayern Munich Victories

```
ForbAll> db.teams.updateOne(
|   { name: "Bayern Munich" },
|   { $set: { "stats.wins": 23 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ForbAll> db.teams.find({ name : "Bayern Munich"}, { _id : 0 , "stats" : 1, name : 1})
[
  {
    name: 'Bayern Munich',
    stats: {
      wins: 23,
      draws: 4,
      losses: 1,
      goals_in_favor: 97,
      goals_against: 25
    }
  }
]
```

2. Add a winner to the Bundesliga Tournament

```
ForbAll> db.tournaments.updateOne(
|   { name: "Bundesliga" },
|   { $set: { winner_team_id: ObjectId("69cb1d23fd64ff05baa71bbe") } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ForbAll> db.tournaments.find({name : "Bundesliga"}, { name : 1, _id : 1, winner_team_id : 1})
[
  {
    _id: ObjectId('69cb1d23fd64ff05baa71be9'),
    name: 'Bundesliga',
    winner_team_id: ObjectId('69cb1d23fd64ff05baa71bbe')
  }
]
```

3. Change the MVP of a match

```
ForbAll> db.matches.updateOne(
|   { _id: ObjectId("69cb1d23fd64ff05baa71c3a") },
|   { $set: { mvp_player_id: ObjectId("69cb1d23fd64ff05baa71bf1") } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. Increment the goals of Bayern in one match

```
ForbAll>  db.matches.updateOne(
|   { "home_team.name": "Bayern Munich" },
|   { $inc: { "home_team.goals": 1 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Change the team of a player

```
ForbAll> db.people.updateOne(
|   { first_name: "Jamal" },
|   { $set: { team_id: ObjectId("69cb1d23fd64ff05baa71bbf") } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5 Deletions

1. Delete a specific Match

```
ForbAll> db.matches.deleteOne(
|   { _id: ObjectId("69cb1d23fd64ff05baa71c3b") }
| )
{ acknowledged: true, deletedCount: 1 }
```

2. Delete all matches of match_day = 2

```
ForbAll> db.matches.deleteMany(
|   { match_day: 2 }
| )
{ acknowledged: true, deletedCount: 1 }
```

3. Delete players of Bayern

```
ForbAll> db.people.deleteMany(
|   { team_id: ObjectId("69cb1d23fd64ff05baa71bbe") }
| )
{ acknowledged: true, deletedCount: 6 }
```

4. Delete teams with low wins.

```
ForbAll> db.teams.deleteMany(
|   { "stats.wins": { $lt: 5 } }
| )
{ acknowledged: true, deletedCount: 2 }
```

5. Delete tournaments without winner.

```
ForbAll> db.tournaments.deleteMany(
|   { winner_team_id: null }
| )
{ acknowledged: true, deletedCount: 3 }
```

Step 7 — Reflection: Performance & Design

Which queries benefit the most from indexes?
The queries performed on text fields, either partial or complete matches, benefit the most from indexes. Without indexes on those fields, the search would be done in a linear way, which directly impacts performance as the data grows.

Which queries are still inefficient?
Queries performed on text fields that are not indexed are still inefficient, since they require scanning the entire collection.

Did your Lab 1 design need changes? Why?
My Lab 1 design needed multiple changes, especially in the bi-directional way the data was stored. Initially, I thought that storing references between collections in both directions would make querying by certain parameters easier.

However, while working on this second lab, I realized that the idea itself was not wrong, but I didn't fully consider the trade-offs. Having multiple collections referencing each other increases the complexity of maintaining consistency, since now the responsibility of updating data correctly in multiple places moves to the application layer.

This makes the backend more complex, as more logic is needed to validate, allow, or deny operations in the database.

What would you change for a production system?
For a production system, I would change the structure of the services, validators, and how the output is processed, since currently there are inefficient mappings being done at the application layer.

I would also think more from a business perspective, focusing on the primary users of the system to better understand what kind of data they need and how they will retrieve it. This would help me make better decisions about where to create indexes and when it is better to use references between collections.

Personal Reflection
In a more personal reflection, I liked this lab. At the beginning, I saw it as a great challenge, not only to apply what we learned in class, but also what I have learned in Software Development I, trying to structure the project in a more backend-oriented way by separating responsibilities and modularizing the components.

However, as the complexity started to grow, I began to feel uncertain about whether I would be able to finish the lab. When time started to play against me, I decided to use AI as an assistant to suggest code (lines or methods), and based on my own understanding, I would accept, reject, or modify those suggestions.

Although I used a significant amount of AI-generated code, I did not accept any piece without understanding what it was doing. The main difficulty was not the complexity of the code itself, but rather the amount of validations and mappings between resources, IDs, names, and relationships that can be seen in the codebase.

If I were to start over, I would begin with simpler foundations, still aiming for a good backend structure, but limiting unnecessary mappings. I would also try to better define my end goal from the beginning instead of figuring it out as I progress, and most importantly, start earlier. Even though I didn’t start at the last moment, I could have started from day 0.